# Paper Critique: Attention Is All You Need

**Full Name:** Mathis DOUTRE
**Student ID:** 904-04-8015
**Paper #:** 9
**Title of the paper you choose to critique:** Attention Is All You Need

---

This critique situates the Transformer in the evolution of seq2seq models, explaining its break from recurrent and convolutional designs, and outlining weaknesses later addressed by follow-up work.

## 1 Problems in Older Seq2Seq Models

### 1.1 Problem: The sequential bottleneck in prior seq2seq models

**Core issue.** Prevailing encoder–decoder models (RNNs and CNNs) are constrained by **time-ordered computation**. In recurrent Neural Machine Translation (NMT) (e.g., GRU/LSTM), each hidden state depends on the previous one, forcing **sequential processing**: no within-example parallelism, longer training time, and reduced batch sizes as sequences grow. "Attention" improved alignment but, embedded inside RNN encoder–decoders (e.g., Bahdanau–Cho–Bengio), it **did not remove the recursive time loop** [1].

**Why CNNs don't fully fix it & what the Transformer changes.** Convolutional seq2seq models compute positions in parallel, yet must **stack many layers to expand receptive fields**; connecting distant tokens still requires multiple steps [5]. Each convolution only mixes information within its kernel (width $k$), to link tokens $n$ apart you still need $O\big(n/(k-1)\big)$ layers/steps (dilations help but still require multiple steps), whereas self-attention connects any pair in a single layer. In short, prior seq2seq systems either **serialize computation** (RNNs) or **need deep stacks to span long contexts** (CNNs), both limiting efficiency on long sequences. The Transformer addresses this bottleneck by removing recurrence and fixed convolutions, enabling fully parallel token interactions via self-attention.

### 1.2 Distance vs. compute (one view of why long-range is hard)

The paper frames long-range difficulty with two linked quantities: (i) the **path length**, the minimum number of sequential operations between two positions, and (ii) the **distance-dependent compute** required to couple far tokens. In RNNs, both the path and the operation count grow linearly with sequence length $n$: information from token 1 to token $n$ must traverse $O(n)$ recurrent steps, which are inherently non-parallel within an example, weakening gradients and increasing latency.

Convolutional encoder–decoders remove the strict time loop (full layer-wise parallelism), but a single layer only mixes local neighborhoods; relating distant tokens still requires depth, roughly linear in kernel span for ConvS2S [5] and logarithmic via dilations for ByteNet [6]. In practice, longer effective paths make optimization and credit assignment harder, while spanning larger distances

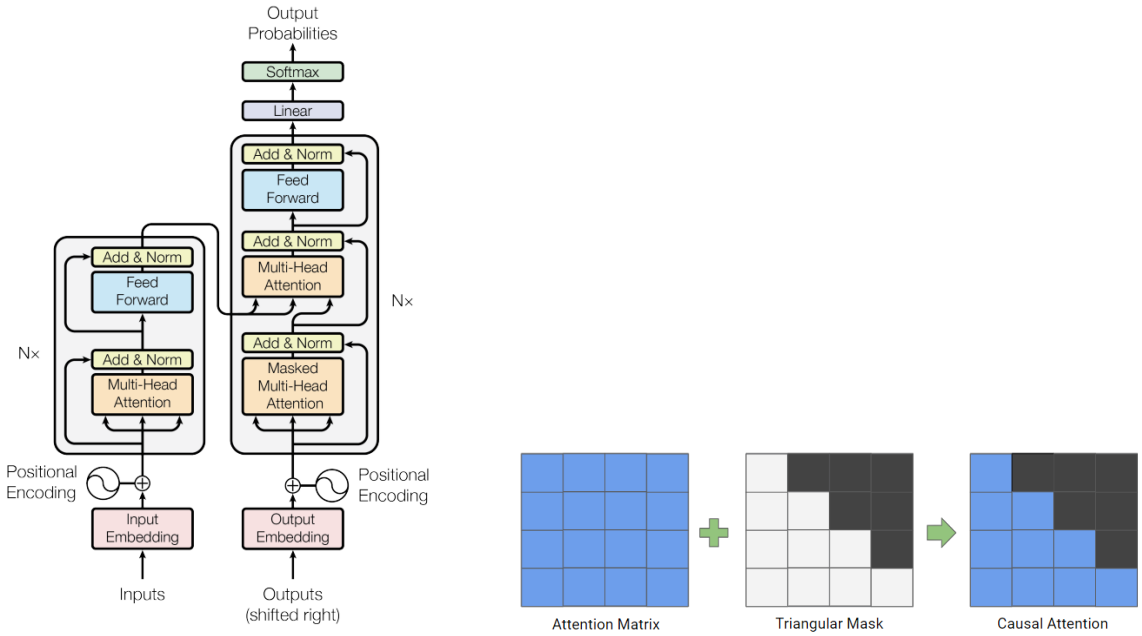
increases FLOPs, parameters, and memory, so "far tokens can talk" only through deeper or more complex stacks. This sets up the Transformer's idea: use **self-attention** to achieve **constant path length** (one hop) between any pair of tokens [9].

# 2 Core Contributions & Strengths

## 2.1 Architectural break: attention instead of recurrence/convolutions

The Transformer replaces recurrence and convolution with *self-attention*, allowing every token to use information from any other token in parallel. During training there is no time-step loop: the encoder processes the entire source at once, and the decoder attends to earlier target positions (teacher-forced during training) and to the encoded source rather than propagating a recurrent state. During training, all target positions are computed in parallel under a causal mask (Fig. 1b); during inference, decoding remains left-to-right and sequential. The repeated block (multi-head self-attention followed by a position-wise feed-forward network, both with residual connections and LayerNorm) shortens the effective path between distant tokens to one attention hop *per layer* and, together with positional encodings, eliminates sequential dependencies within a training example in the encoder.

Causal masking blocks attention to future positions, preserving autoregression while keeping training as batched matrix multiplications over the visible context. In practice, encoder self-attention captures long-range structure, e.g., in Fig 2c, multiple heads from the token "making" attending to a distant complement completing "making ... more difficult," with some heads specializing in local patterns and others in long-distance relations.




(a) Transformer stack: encoder = self-attention + FFN; decoder adds causal mask and cross-attention. Residuals and LayerNorm wrap sublayers [9].

(b) Causal mask: upper-triangular mask blocks future tokens, preserving left-to-right decoding while enabling parallel training [11].

Figure 1: Transformer stack (left) and causal masking (right).

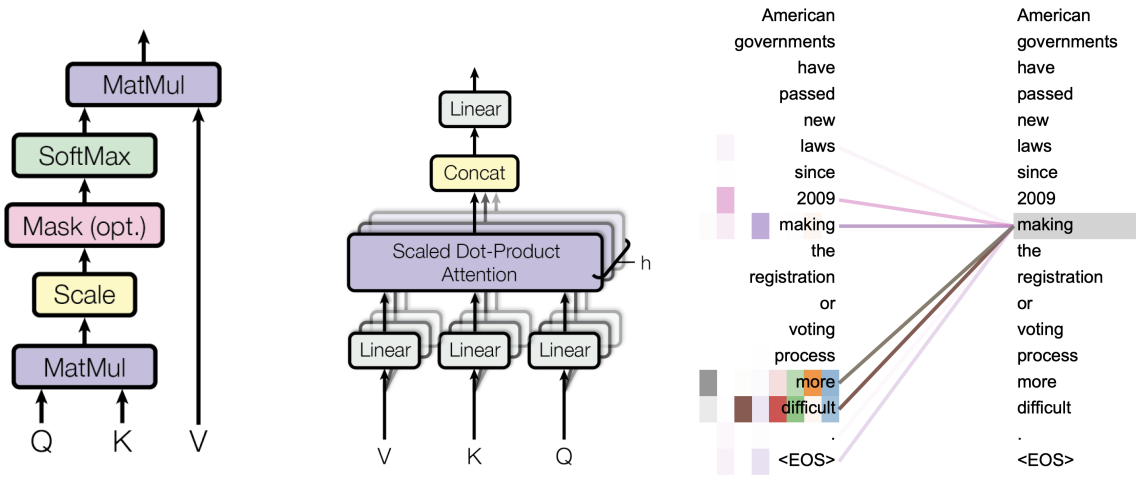
### 2.2 Scaled dot-product attention and multi-head attention

Each token forms three small vectors: a *query* ($Q$) that asks what it is looking for, a *key* ($K$) that says what it offers, and a *value* ($V$) that is the content mixed if weighted. To update a token, we compare its query to every other token's key using dot products, scale the scores by $1/\sqrt{d_k}$ to prevent softmax saturation from large dot products, optionally apply a mask by setting forbidden logits to $-\infty$, convert the scores to weights with softmax, and take a weighted sum of the values (See Fig 2c):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \text{mask}\right) V.$$

Self-attention creates a constant path between any two tokens, one hop per layer, so dependency difficulty does not grow with token distance, even though compute and memory still scale as $\mathcal{O}(n^2)$. With a causal mask in the decoder, training proceeds as batched matrix multiplications over the visible context, while inference remains left-to-right and sequential.

Rather than computing a single attention, the Transformer runs several in parallel on different learned projections of $Q$, $K$, and $V$. For head $i$, $Q_i = QW_i^Q$, $K_i = KW_i^K$, $V_i = VW_i^V$. Each head performs the same scaled attention, then the head outputs are concatenated and multiplied by $W^O$ (multi-head attention). Different heads specialize in different relations, some local and others long-distance, without adding sequential depth.




(a) Scaled dot-product: $\text{softmax}(QK^\top/\sqrt{d_k})V$. *Adapted from* [9].

(b) Multi-head: parallel heads $\rightarrow$ Concat $\rightarrow W^O$. *Adapted from* [9].

(c) Example long-range encoder heads. *Adapted from* [9].

Figure 2: Scaled dot-product and multi-head attention with a long-range pattern.

### 2.3 Other blocks

The position-wise feed-forward network (FFN) increases per-token capacity independently of sequence mixing, so attention is not merely a linear mixer. Concretely, the same two-layer MLP is applied at every position:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)\, W_2 + b_2,$$

with a width expansion from $d_{\text{model}}$ to $d_{\text{ff}}$ (Base uses $512 \rightarrow 2048$), providing nonlinearity and channel re-mixing after token mixing by attention.

Residual connections and LayerNorm wrap each sublayer to improve gradient flow and training stability in deep stacks, making an attention-based architecture expressive and trainable. In the original configuration, LayerNorm is applied after the residual addition (post-norm), and dropout is used on attention weights, within the FFN, and on sublayer outputs before the residual path.

### 2.4 Speed and quality in practice

**Why it trains fast.** The Transformer removes the time-step loop: encoder self-attention and teacher-forced decoder self-attention operate over full sequences, so training is fully parallel across positions. Because self-attention gives a constant per-layer path between any two tokens, long-range structure is easier to learn. Although attention is $O(n^2)$, WMT sentence lengths (tens of tokens) keep it manageable, and parallelism still dominates.

**Why it translates well.** Decoder cross-attention gives each target position direct access to any source token, avoiding the alignment indirection typical of recurrent encoders. Multi-head attention lets different heads specialize in complementary relations, some local and syntactic, others long-range and semantic (See Fig 2b). A practical training recipe with Adam, warmup, label smoothing, and dropout trains reliably and quickly on these tasks.

**Inference reality.** Decoding remains left-to-right and sequential across output steps. Beam search with modest settings keeps runtime competitive.

### 2.5 Comparison to prior state of the art (and training time)

**Headline results, single models.**
*WMT'14 En→De:* 28.4 BLEU with Transformer-Big.
*WMT'14 En→Fr:* 41.0 BLEU with Transformer-Big.

These results beat the best single models and rivaled or surpassed strong ensembles of the time. On WMT'14 En→De, the prior single-model SOTA was about 26.03 BLEU with a mixture-of-experts system [8], and top ensembles were around 26.3–26.4 [8, 5, 6]. On WMT'14 En→Fr, the prior single-model SOTA was about 40.5–40.6 with ConvS2S or MoE [5, 8], and ensembles were around 41.2–41.3 [5]. The Transformer-Big results exceeded single-model baselines and came close to, or exceeded, these results.

**Compute and time budget.** On a single machine with 8× NVIDIA P100 GPUs, Transformer-Base (6 encoder and 6 decoder layers, $d_{\text{model}}$=512, 8 heads) reaches its reported quality in about 12 hours, roughly 100k steps. Transformer-Big ($d_{\text{model}}$=1024, 16 heads) reaches its reported quality in about 3.5 days, roughly 300k steps. Even Base outperforms earlier single-model baselines at similar or lower training cost.

**Why it mattered.** The Transformer established a new Pareto point : higher BLEU with less wall-clock training on standard multi-GPU hardware, using comparable datasets and evaluation, with gains attributable to the attention plus FFN design rather than heavy engineering.

## 3 Key Weaknesses and Limitations & Suggestions

### 3.1 Attention cost on long sequences (training and inference)

**Weakness.** Full self-attention computes an $n \times n$ score matrix at training time, so time and memory scale as $\mathcal{O}(n^2)$. At inference, decoding is autoregressive and sequential; even with key/value caching the wall-clock latency grows with output length, and the cache itself scales as $\mathcal{O}(nd)$.

**Why it matters.** For long documents, multi-turn contexts, or code, quadratic training cost reduces batch size and slows experiments, while decode latency and cache memory become the practical bottlenecks in production.

**Suggestions.** Use *efficient attention* where quality permits: block- and global-sparse patterns (*Longformer*, *BigBird*) reduce effective pairs; low-rank or kernel projections (*Linformer*) compress interactions; hashing with reversible layers (*Reformer*) lowers compute and activation memory [2, 12, 10, 7]. A pragmatic recipe is to mix layers: keep a few full-attention layers for global integration and use efficient attention in the rest. For latency, combine KV caching with speculative decoding and cache chunking to improve locality.

### 3.2 Context length and position encoding

**Weakness.** Vanilla Transformers use a fixed context window and absolute positional encodings. Dependencies that cross window boundaries must be re-encoded, and generalization can degrade when sequence length exceeds what was seen during training.

**Why it matters.** Long-range tasks (long-form QA, long code functions) suffer from truncation and poor length extrapolation, which shows up as abrupt failures at segment boundaries or declining quality at greater lengths.

**Suggestions.** Adopt segment-level memory with *relative* positions so information flows across segments without recomputing the entire prefix. *Transformer-XL* demonstrates this and improves long-range modeling [3]. In practice, set the memory length to the expected dependency span and validate on sequences longer than the training range. If you cannot change the architecture, constrain inputs with retrieval or summarization to keep critical context in-window.

### 3.3 Inductive bias and data efficiency

**Weakness.** Given $Q$ and $K$, the attention update is linear in $V$ and does not encode locality or hierarchy by default. With limited data, this weaker inductive bias can reduce sample efficiency relative to architectures with stronger sequence priors.

**Why it matters.** Low-resource languages, morphology-heavy tasks, or structured domains can underperform unless the model is helped to focus on local patterns and compositional structure.

**Suggestions.** Inject light bias without giving up parallel training: use windowed attention with a small set of global tokens, relative positions, and small convolutional or gated modules inside the FFN; weight sharing or modest recurrence as in the *Universal Transformer* adds iterative refinement while preserving parallelizable training [4]. When data is scarce, strengthen regularization and start from shorter-span curricula before exposing full lengths.

## References


[1] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate, 2016. URL https://arxiv.org/abs/1409.0473.

[2] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

[3] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context, 2019. URL https://arxiv.org/abs/1901.02860.

[4] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and Łukasz Kaiser. Universal transformers, 2019. URL https://arxiv.org/abs/1807.03819.

[5] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin. Convolutional sequence to sequence learning, 2017. URL https://arxiv.org/abs/1705.03122.

[6] N. Kalchbrenner, L. Espeholt, K. Simonyan, A. van den Oord, A. Graves, and K. Kavukcuoglu. Neural machine translation in linear time, 2017. URL https://arxiv.org/abs/1610.10099.

[7] N. Kitaev, Łukasz Kaiser, and A. Levskaya. Reformer: The efficient transformer, 2020. URL https://arxiv.org/abs/2001.04451.

[8] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017. URL https://arxiv.org/abs/1701.06538.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://papers.nips.cc/paper/7181-attention-is-all-you-need.

[10] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity, 2020. URL https://arxiv.org/abs/2006.04768.

[11] S. Yadav. Building a transformer/llm with code: Fundamental transformer (gpt). https://www.yadavsaurabh.com/building-a-transformer-llm-with-code-fundamental-transformer-gpt, 2024. Accessed 2025-09-18.

[12] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed. Big bird: Transformers for longer sequences, 2021. URL https://arxiv.org/abs/2007.14062.